Encryption/Decryption using RC4

Mohammad Noman
Email: mohammad.noman@ucp.edu.pk

Types of Encryption

- **Private/Symmetric Key Cryptography** :

Same key is used for encryption and decryption.

- **Public/Asymmetric Key Cryptography :**

Different keys are used for encryption and decryption.

RC4 Basics

Stream Cipher vs. Block Cipher

- **Stream Cipher**: Encrypts plaintext one byte at a time, producing a stream of ciphertext.
- **Block Cipher**: Encrypts data in fixed-size blocks (e.g., 128 bits).

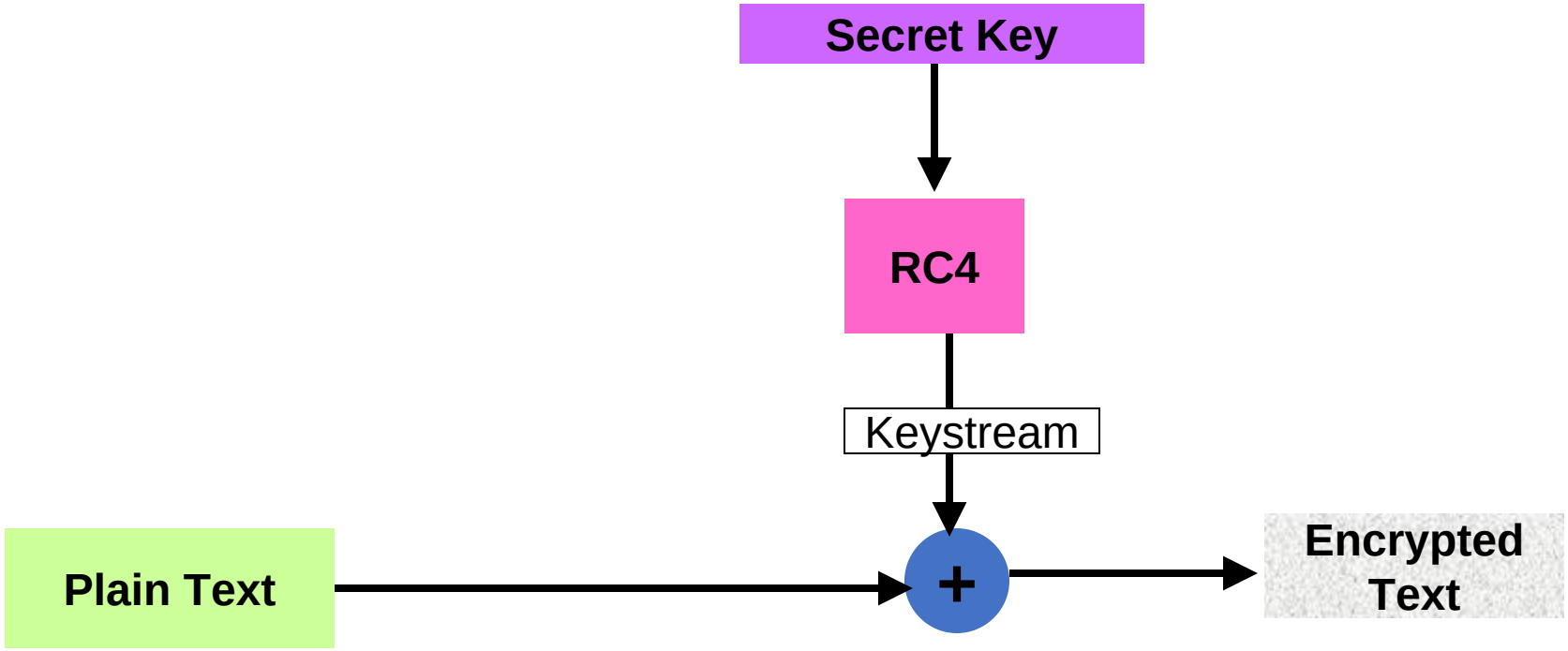
Key Features of RC4

- Variable key length (1 to 256 bytes).
- Fast encryption and decryption processes.

RC4 Basics

- A symmetric key encryption algo. Invented by **Ron Rivest.**
- Most popular implementation is in WEP for 802.11 wireless networks and in SSL.
- Cryptographically very strong yet very easy to implement.
- Consists of 2 parts:
  - Key Scheduling Algorithm (KSA)
  - Pseudo-Random Generation Algorithm (PRGA)

RC4 Block Diagram



RC4 …break up

- Initialize an array of 256 bytes.
- Run the KSA on them
- Run the PRGA on the KSA output to generate keystream.
- XOR the data with the keystream.

Array Initialization

C Code:

```
char S[256];
int i;
For(i=0; i< 256; i++)
        S[i] = i;
```

After this the array would like this :

S[] = { 0,1,2,3, ……, 254, 255}

T[i]=K[i **mod** keylen]

T is temporary array made with secret key.

The KSA

- The initialized array S[256] is now run through the KSA. The KSA uses the secret key to scramble the array.
- C Code for KSA:

```
int  i, j = 0;
for(i=0; i<256; i++)
{
      j = ( j + S[i] + T[i]) % 256;
      swap(S[i], S[j]);
}
```

The PRGA

- The KSA scrambled S[256] array is used to generate the PRGA. This is the actual keystream.
- C Code:

```
i = j = 0;
while(true)
{
        i = ( i + 1) % 256;
        j = ( j + S[i] ) % 256;
        swap( S[i], S[j] );
        output = S[ ( S[i] + S[j] ) % 256 ]
}
```

Encryption using RC4

- Choose a secret key
- Run the KSA and PRGA using the key to generate a keystream.
- XOR keystream with the data to generated encrypted stream.
- Transmit Encrypted stream.

Decryption using RC4

- Use the same secret key as during the encryption phase.
- Generate keystream by running the KSA and PRGA.
- XOR keystream with the encrypted text to generate the plain text.
- Logic is simple :

$$(A \text{ XOR } B) \text{ XOR } B = A$$

A = Plain Text or Data
B = KeyStream

Example

Scenario Key:
"Key"
(ASCII values [75, 101, 121])

Plaintext:
"HELLO"
(ASCII values [72, 69, 76, 76, 79])

Making of a RC4 File Encryptor

- Using a secret key generate the RC4 keystream using the KSA and PRGA.
- Read the file and XOR each byte of the file with the corresponding keystream byte.
- Write this encrypted output to a file.
- Transmit file over an insecure channel.

Summary

- **Key-Scheduling Algorithm (KSA)** sets up the state array S.
- **Pseudo-Random Generation Algorithm (PRGA)** generates the keystream.
- **Encryption** is performed by XORing the plaintext with the keystream.
- **Decryption** is done by XORing the ciphertext with the same keystream.